

VISION

To be recognized for excellent engineering, developing global leaders both in educational and research in the domain of computer science and wireless engineering.

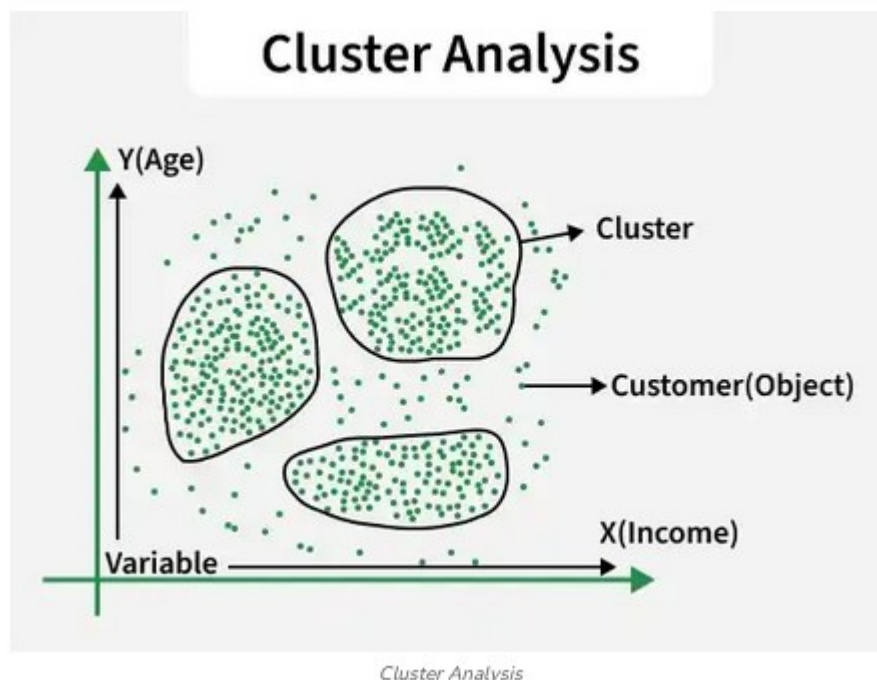
MISSION

1. To create self-learning environment by facilitating leadership qualities, team spirit and ethical responsibilities.
2. To improve department-industry collaboration, interaction with professional society through technical knowledge and internship program.
3. To promote research and development with current techniques through well qualified resources in the area of computer science and wireless engineering.

UNIT-04

Cluster Analysis:

- Cluster analysis is also known as clustering, which groups similar data points forming clusters.
- The goal is to ensure that data points within a cluster are more similar to each other than to those in other clusters.
- For example, in e-commerce retailers use clustering to group customers based on their purchasing habits. If one group frequently buys fitness gear while another prefers electronics. This helps companies to give personalized recommendations and improve customer experience.



Types of Data in Cluster Analysis

- 1) Interval-Scaled variables
- 2) Binary variables

- 3) **Nominal**
- 4) **Ordinal**
- 5) **Ratio variables**
- 6) **Variables of mixed types**

1) Interval-Scaled variables:

- These are quantitative measurements (continuous or discrete) where the distance between values matters (e.g., age, weight, temperature).
- The measurement unit used can affect the clustering analysis. For example, changing measurement units from meters to inches for height, or from kilograms to pounds for weight, may lead to a very different clustering structure.
- In general, expressing a variable in smaller units will lead to a larger range for that variable, and thus a larger effect on the resulting clustering structure.
- **Examples:** Height: 170cm, 180cm; Temperature: 25°C, 30°C.

2) Binary variables:

- These variables only have two states (0 or 1, true or false, Yes or No).
- A binary variable is a variable that can take only 2 values.

Symmetric Binary: Both states are equally important (e.g., Male/Female in general analysis).

Asymmetric Binary: One state is more important (usually 1) and rare (e.g., Test result Positive/Negative, Disease Present/Absent).

3) Nominal:

- These variables represent distinct categories without any inherent order.
- A generalization of the binary variable in that it can take more than 2 states
- Examples: Colors (Red, Blue, Green), Hair Type (Curly, Straight), City.

4) Ordinal:

- These are categorical variables where the order or ranking matters, but the exact difference between values is unknown.
- Examples: Academic grade (A, B, C), Satisfaction survey (Poor, Fair, Good, Excellent).

5) Ratio variables:

- A special type of numerical data that has a true zero point, allowing ratio comparisons (e.g., price, population, time).
- Similar to interval, but sometimes scaled logarithmically if the data spans several orders of magnitude.
- **Example:** Age, Income, Density.

6) Mixed Data Types:

- A database may contain all the six types of variables symmetric binary, asymmetric binary, nominal, ordinal, interval, and ratio and those combinedly called as mixed-type variables.
- Method: Gower's coefficient is commonly used, which computes a custom similarity score for each attribute type and combines them.

Partitioning Method:

Partitioning Methods divide the data into k groups (clusters) where each data point belongs to only one group. These methods are used when you already know how many clusters you want to create.

This clustering method classifies the information into multiple groups based on the characteristics and similarity of the data. It's the data analysts to specify the number of clusters that has to be generated for the clustering methods.

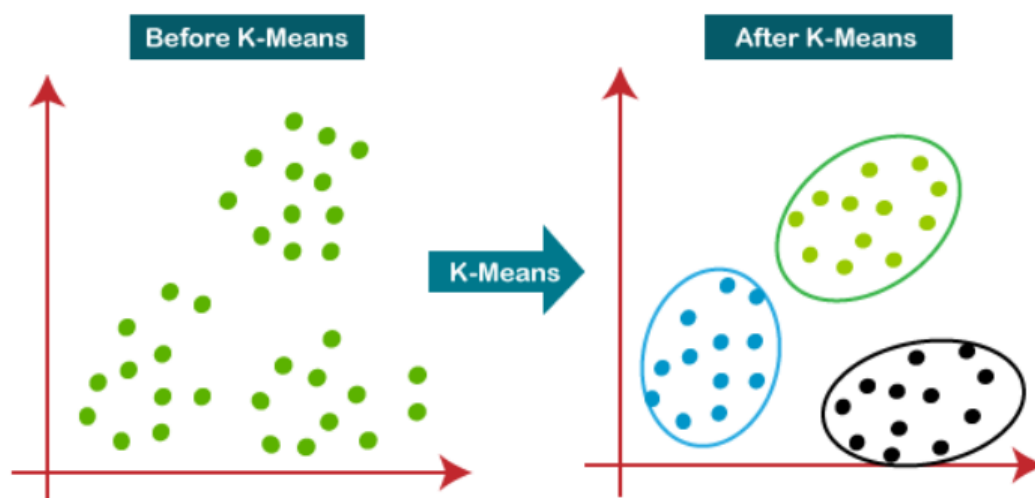
A common example are K-means clustering and K-medoid Clustering.

1. K-Means Clustering:

- K-Means Clustering, which groups the unlabelled dataset into different clusters.
- Here K defines the number of pre-defined clusters that need to be created in the process, as if $K=2$, there will be two clusters, and for $K=3$, there will be three clusters, and so on.
- It is an iterative algorithm that divides the unlabelled dataset into k different clusters in such a way that each dataset belongs only one group that has similar properties.
- It is a centroid-based algorithm, where each cluster is associated with a centroid.
- The main aim of this algorithm is to minimize the sum of distances between the data point and their corresponding clusters.
- The algorithm takes the unlabelled dataset as input, divides the dataset into k -number of clusters, and repeats the process until it does not find the best clusters.
- The value of k should be predetermined in this algorithm.
- The k -means clustering algorithm mainly performs two tasks:
 - o Determines the best value for K center points or centroids by an iterative process.
 - o Assigns each data point to its closest k -center. Those data points which are near to the particular k -center, create a cluster.

Hence each cluster has data points with some commonalities, and it is away from other clusters.

The below diagram explains the working of the K-means Clustering Algorithm:



How does the K-Means Algorithm Work?

The working of the K-Means algorithm is explained in the below steps:

Step-1: Select the number K to decide the number of clusters.

Step-2: Select random K points or centroids. (It can be other from the input dataset).

Step-3: Assign each data point to their closest centroid, which will form the predefined K clusters.

Step-4: Calculate the variance and place a new centroid of each cluster.

Step-5: Repeat the third steps, which means reassign each datapoint to the new closest centroid of each cluster.

Step-6: If any reassignment occurs, then go to step-4 else go to FINISH.

Step-7: The model is ready.

K-Medoids Clustering

- K-medoids Clustering is used to group data into different clusters.
- K-medoids is also known as partitioning around medoids (PAM) that partitions a dataset into K-Cluster by Selecting actual data Points, called medoids, as a Cluster Centers.
- It is similar to K-means, but instead of using the mean of points as a cluster center, it uses an actual data point called a medoid.
- It is an iterative algorithm that starts by selecting k data points as medoids in a dataset.
- After this, the distance between each data point and the medoids is calculated. Then, the data points are assigned to clusters associated with the medoid at the minimum distance from each data point.
- Here, the medoid is the most centrally located point in the cluster. Once we assign all the data points to the clusters, we calculate the sum of the distance of all the non-medoid data points to the medoid of each cluster. We term the sum of distances as the cost.
- After calculating the cost, we select a temporary non-medoid point randomly from the dataset and swap a medoid with the selected point.
- Then we recalculate the cost of all the non-medoid data points to the medoid of each cluster considering the temporarily selected point as the medoid.
- If the newly calculated cost is less than the previous cost, we make the temporary point the permanent centroid.
- If the new cost is greater than the previous cost, we undo the changes. Then, we again select a non-medoid point and repeat the process until the cost is minimized.

What is a Medoid

A medoid is the most centrally located data point within a cluster. It minimizes the total dissimilarity with all other points in that cluster. The dissimilarity between a medoid C_i and an object P_i is given by: $E=|P_i-C_i|$

The **total cost** (or objective function) of K-Medoids is defined as:

$$c = \sum C_i \sum_{P_i \in C_i} |P_i - C_i|$$

Manhattan distance

The distance between each data point from both medoids is calculated using the **Manhattan distance** formula. It is also known as the cost.

$$\text{Distance} = |x_2 - x_1| + |y_2 - y_1|$$

k-medoids Algorithm:

- **Step-1: Select k random points from the dataset:** First, the algorithm selects k random points from the dataset as the initial medoids. The medoids that are chosen are used to define the initial k clusters.
- **Step-2: Assign data points to the cluster of the nearest medoid:** It then assigns each non-medoid to the cluster that has the closest medoid.
- **Step-3: Calculate the total sum of distances of data points from their assigned medoids for each medoid:** It then calculates the cost, i.e., the total sum of distances or dissimilarities of the data points from the assigned medoid.

$$\text{Distance} = |x_2 - x_1| + |y_2 - y_1|$$

$$\text{Calculate the total Cost } c = \sum C_i \sum_{P_i \in C_i} |P_i - C_i|$$
- **Step-4: Swap a non-medoid point with a medoid point and recalculate the cost:** It then swaps every non-medoid point with the medoid assigned to it and recalculates the total sum of distances.
 - 1) Recalculate the cost for this new configuration.
 - 2) If the total cost decreases, accept the swap; otherwise, revert
- **Step-5: Undo the swap if the recalculated cost with the new medoid exceeds the previous cost:** Finally, it evaluates whether the recalculated cost is more than the previously calculated cost. If this is the case, it undoes the swap, and the algorithm converges. If the recalculated cost is less, it repeats step 4.

Hierarchical methods:

Hierarchical clustering builds a tree-like structure of clusters known as a **dendrogram** that represents the merging or splitting of clusters. It can be divided into:

- **Agglomerative Approach (Bottom-up):** Agglomerative Approach starts with individual points and merges similar ones. Like a family tree where relatives are grouped step by step.
- **Divisive Approach (Top-down):** It starts with one big cluster and splits it repeatedly into smaller clusters. For example, classifying animals into broad categories like mammals, reptiles, etc and further refining them.

Hierarchical clustering

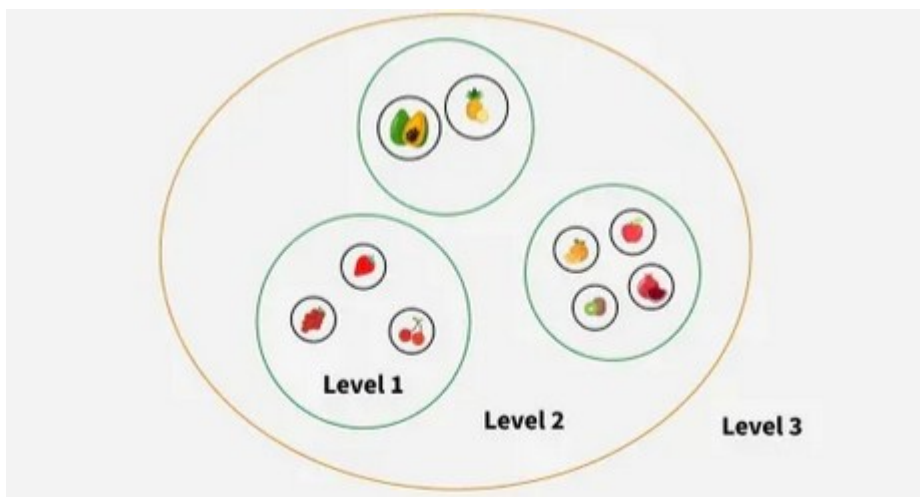
- Hierarchical clustering is an unsupervised learning technique used to group similar data points together based on their Similarity Creating a hierarchy or tree-like structure.
- The Key idea is to begin with each data point as its own Separate Cluster and then Progressively merge or Split them based on their Similarity

Example

Imagine we have four fruits with different weights: an apple (100g), a banana (120g), a cherry (50g) and a grape (30g). Hierarchical clustering starts by treating each fruit as its own group.

- Start with each fruit as its own cluster.
- Merge the closest items: grape (30g) and cherry (50g) are grouped first.
- Next, apple (100g) and banana (120g) are grouped.
- Finally, these two clusters merge into one.

Finally, all the fruits are merged into one large group, showing how hierarchical clustering progressively combines the most similar data points.



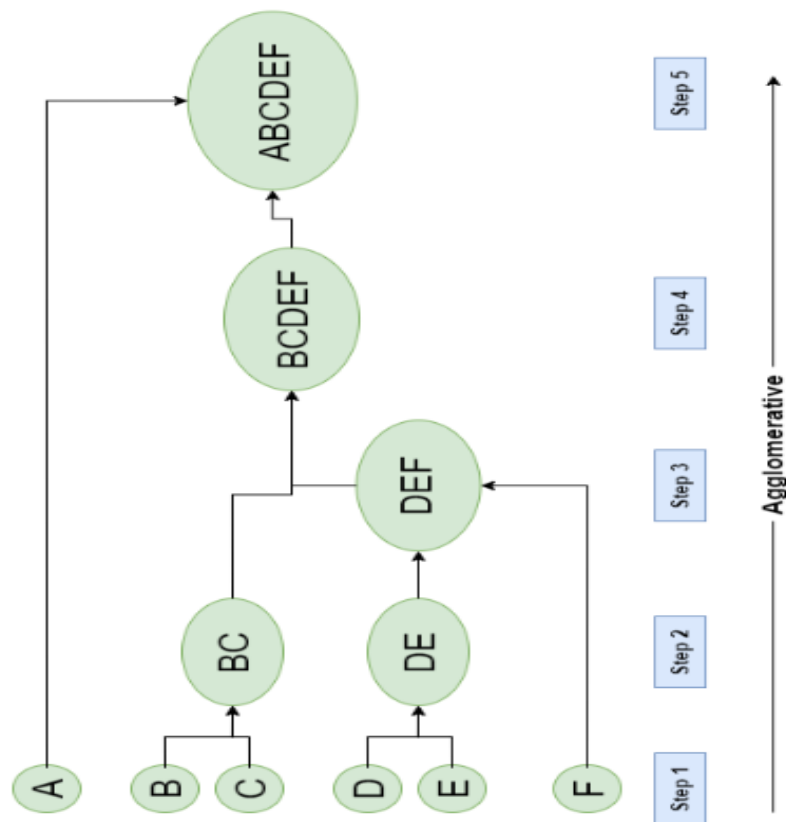
Hierarchical Agglomerative Clustering

- It is also known as the bottom-up approach or hierarchical agglomerative clustering (HAC).
- Unlike flat clustering hierarchical clustering provides a structured way to group data.
- This clustering algorithm does not require us to prespecify the number of clusters.
- Bottom-up algorithms treat each data as a singleton cluster at the outset and then successively agglomerate pairs of clusters until all clusters have been merged into a single cluster that contains all data.

Workflow for Hierarchical Agglomerative clustering

1. **Start with individual points:** Each data point is its own cluster. For example, if you have 5 data points you start with 5 clusters each containing just one data point.
2. **Calculate distances between clusters:** Calculate the distance between every pair of clusters. Initially since each cluster has one point this is the distance between the two data points.
3. **Merge the closest clusters:** Identify the two clusters with the smallest distance and merge them into a single cluster.
4. **Update distance matrix:** After merging you now have one less cluster. Recalculate the distances between the new cluster and the remaining clusters.

5. **Repeat steps 3 and 4:** Keep merging the closest clusters and updating the distance matrix until you have only one cluster left.
6. **Create a dendrogram:** As the process continues you can visualize the merging of clusters using a tree-like diagram called a **dendrogram**. It shows the hierarchy of how clusters are merged.



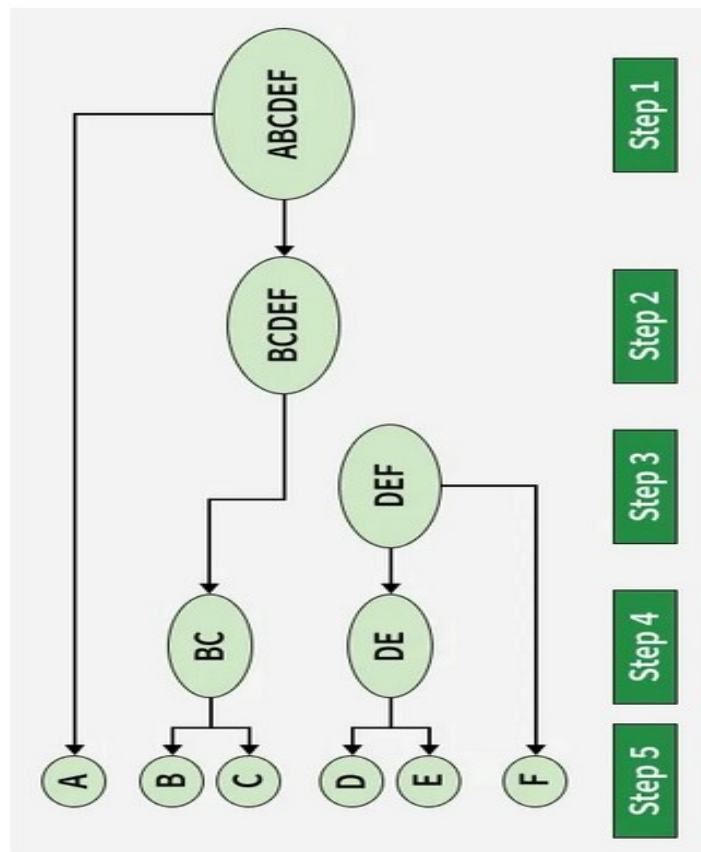
Hierarchical Agglomerative Clustering

2. Hierarchical Divisive clustering

Divisive clustering is also known as a top-down approach. Top-down clustering requires a method for splitting a cluster that contains the whole data and proceeds by splitting clusters recursively until individual data have been split into singleton clusters.

Workflow for Hierarchical Divisive clustering:

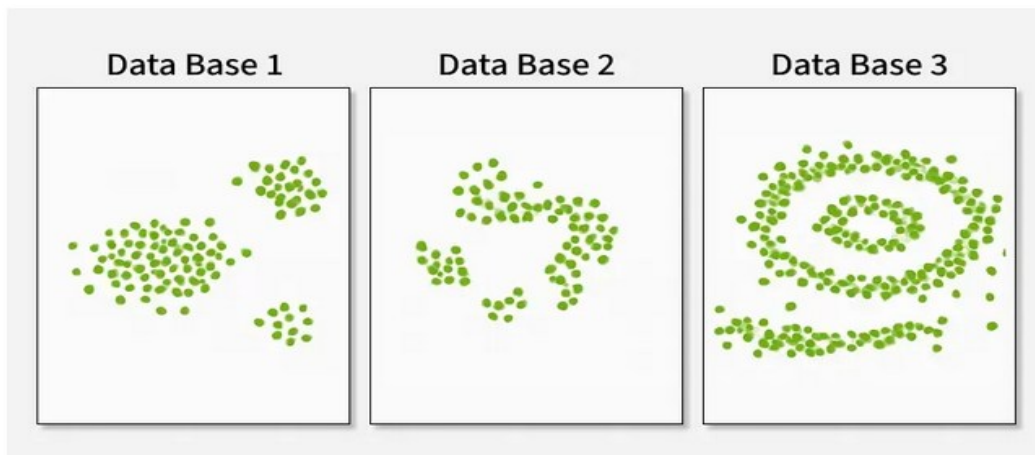
1. **Start with all data points in one cluster:** Treat the entire dataset as a single large cluster.
2. **Split the cluster:** Divide the cluster into two smaller clusters. The division is typically done by finding the two most dissimilar points in the cluster and using them to separate the data into two parts.
3. **Repeat the process:** For each of the new clusters, repeat the splitting process: Choose the cluster with the most dissimilar points and split it again into two smaller clusters.
4. **Stop when each data point is in its own cluster:** Continue this process until every data point is its own cluster or the stopping condition (such as a predefined number of clusters) is met.



Divisive Clustering

Density-Based Methods – DBSCAN Clustering

- Density-based clustering is an unsupervised machine learning technique that groups data points by identifying contiguous regions of high point density.
- DBSCAN is a density-based clustering algorithm that groups data points that are closely packed together and marks outliers as noise based on their density in the feature space.
- It identifies clusters as dense regions in the data space separated by areas of lower density.
- Unlike K-Means or hierarchical clustering which assumes clusters are compact and spherical, DBSCAN perform well in handling real-world data irregularities such as:
 - **Arbitrary-Shaped Clusters:** Clusters can take any shape not just circular or convex.
 - **Noise and Outliers:** It effectively identifies and handles noise points without assigning them to any cluster.



The figure above shows a data set with clustering algorithms: K-Means and Hierarchical handling compact, spherical clusters with varying noise tolerance while DBSCAN manages arbitrary-shaped clusters and noise handling.

Key Parameters in DBSCAN

1. eps: This defines the radius of the neighborhood around a data point. If the distance between two points is less than or equal to eps they are considered neighbors. A common method to determine eps is by analyzing the k-distance graph. Choosing the right eps is important:

- If eps is too small most points will be classified as noise.
- If eps is too large clusters may merge and the algorithm may fail to distinguish between them.

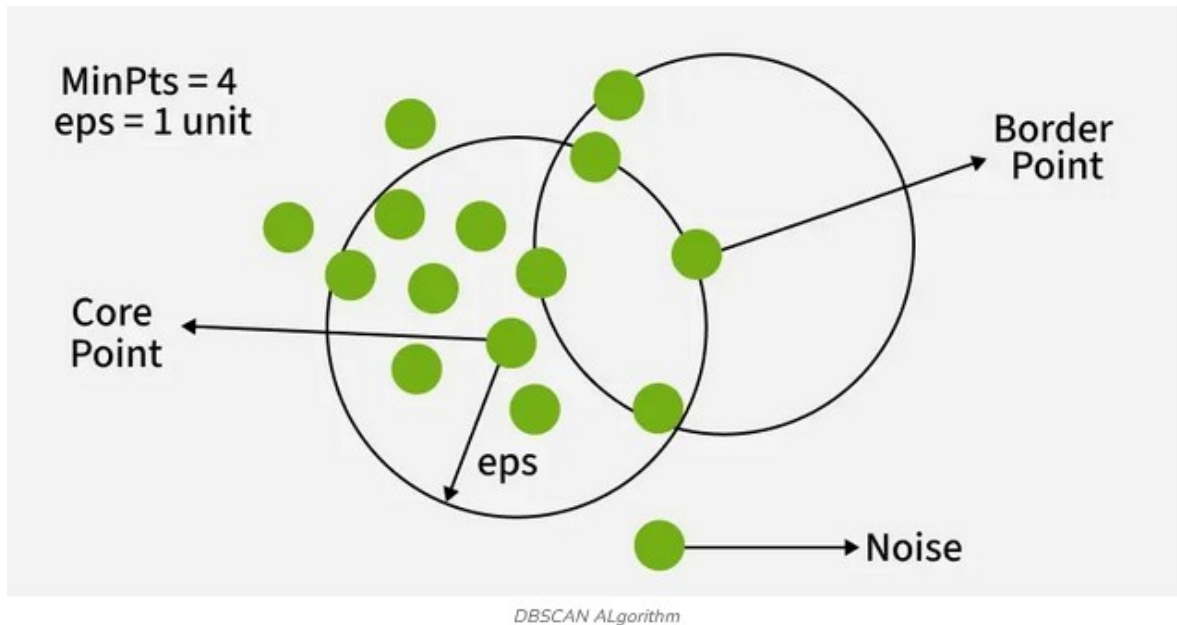
2. MinPts: This is the minimum number of points required within the **eps** radius to form a dense region. A general rule of thumb is to set $\text{MinPts} \geq D+1$ where **D** is the number of dimensions in the dataset.

For most cases a minimum value of **MinPts = 3** is recommended.

DBSCAN works by categorizing data points into three types:

- 1. Core Points:** A point is a "core" if it has at least a minimum number of neighbors (MinPts) within a specified radius (Epsilon/ ϵ).
- 2. Border Points:** These lie within the radius of a core point but do not have enough neighbors themselves to be considered core points.
- 3. Noise Points (Outliers):** Points that are neither core nor border points. Noise points which do not belong to any cluster.

By iteratively expanding clusters from core points and connecting density-reachable points, DBSCAN forms clusters without relying on rigid assumptions about their shape or size.



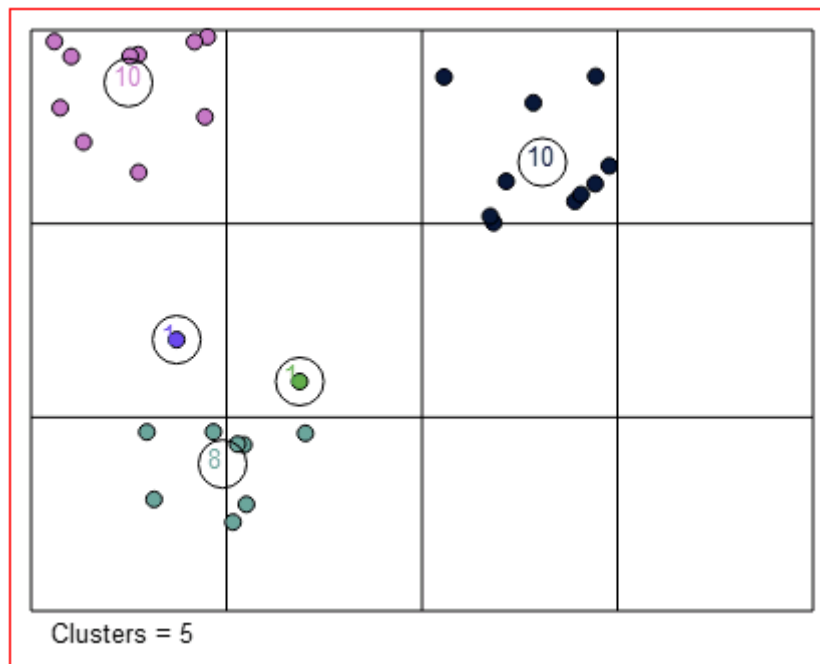
Steps in the DBSCAN Algorithm

1. **Identify Core Points:** For each point in the dataset count the number of points within its eps neighborhood. If the count meets or exceeds MinPts mark the point as a core point.
2. **Form Clusters:** For each core point that is not already assigned to a cluster create a new cluster. Recursively find all density-connected points i.e points within the eps radius of the core point and add them to the cluster.
3. **Density Connectivity:** Two points a and b are density-connected if there exists a chain of points where each point is within the eps radius of the next and at least one point in the chain is a core point. This chaining process ensures that all points in a cluster are connected through a series of dense regions.
4. **Label Noise Points:** After processing all points any point that does not belong to a cluster is labeled as noise.

Grid-Based Methods:

- Grid-based clustering methods in data mining quantize the data space into a finite number of cells (a grid structure) and perform clustering on these cells rather than individual data points.
- The grid-based clustering methods use a multi-resolution grid data structure.
- It quantizes the object areas into a finite number of cells that form a grid structure on which all of the operations for clustering are implemented.
- The benefit of the method is its quick processing time, which is generally independent of the number of data objects, still dependent on only the multiple cells in each dimension in the quantized space.
- This approach is highly efficient for large datasets, as processing time depends on the number of grid cells rather than the number of data objects.
- An instance of the grid-based approach involves STING, which explores statistical data stored in the grid cells, WaveCluster, which clusters objects using a wavelet transform approach.

- CLIQUE, which defines a grid-and density-based approach for clustering in high-dimensional data space.
- STING is a grid-based multiresolution clustering method in which the spatial area is divided into rectangular cells.



Outlier Analysis

Outlier is a data object that deviates significantly from the rest of the data objects and behaves in a different manner. They can be caused by measurement or execution errors. The analysis of outlier data is referred to as outlier analysis or outlier mining.

These points—also known as anomalies, exceptions, or noise—don't conform to the general behavior or model of the data, making their detection crucial for accurate analysis and decision-making.

An outlier cannot be termed as a noise or error. Instead, they are suspected of not being generated by the same method as the rest of the data objects.

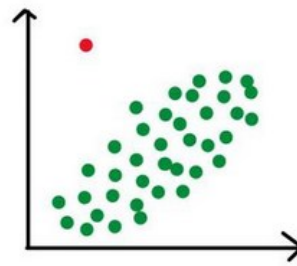
Outliers are of three types, namely -

1. Global (or Point) Outliers
2. Collective Outliers
3. Contextual (or Conditional) Outliers

1. Global (or Point) Outliers:

- Also known as point anomalies, these data point significantly differ from the rest of the dataset.
- Global outliers are data points that deviate significantly from the overall distribution of a dataset.
- Errors in data collection, measurement errors, or truly unusual events can result in global outliers.

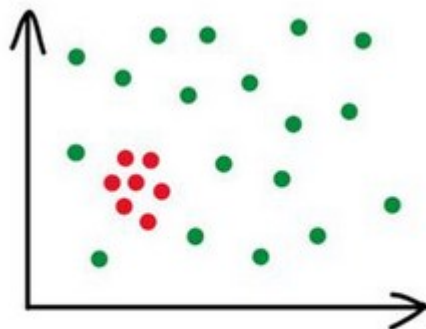
- Global outliers can distort data analysis results and affect machine learning model performance.
- Carefully considering the impact of global outliers is crucial for accurate data analysis and machine learning model outcomes.



The red data point is a global outlier.

2. Collective Outliers:

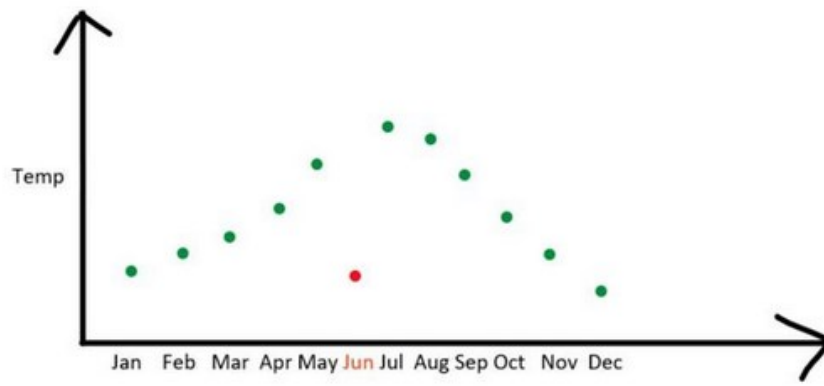
- A collection of data points that deviate significantly from the rest of the dataset, even if individual points within the collection are not outliers.
- Collective outliers may not be outliers when considered individually, but as a group, they exhibit unusual behavior.
- Techniques for detecting collective outliers include clustering algorithms, density-based methods, and subspace-based approaches.
- Collective outliers can represent interesting patterns or anomalies in data that may require special attention or further investigation.



The red data points as a whole are collective outliers.

3. Contextual (or Conditional) Outliers:

- Contextual outliers are data points that deviate significantly from the expected behavior within a specific context or subgroup.
- Contextual outliers may not be outliers when considered in the entire dataset, but they exhibit unusual behavior within a specific context or subgroup.
- Contextual information such as time, location, or other relevant factors are crucial in identifying contextual outliers.
- Contextual outliers can represent unusual or anomalous behavior within a specific context, which may require further investigation or attention.



A low temperature value in June is a contextual outlier because the same value in December is not an outlier.